DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING						
Course Code: 24CS310	Course Title: *Object Oriented Programming using C++*					
Total Credits: 04 L-45: T-0: P-30: SL and TW-45	*SDGs addressed: 4, 8, 9, 12,13*	*Course Category: PCC*	*Course Type: Theory with Practical*			
Teaching Learning Process		*Total Contact Hours/Semester*	*Assessment in Weightage and marks*			
	Lecture	45		CIE	SEE	Total
	Tutorial	-	*Weightage in %*	*40*	*60*	*100*
	Practical	30	*Maximum Marks*	*40*	*60*	*100*
	Self-learning and Term Work	45	*Minimum Marks*	*20*	*25*	*45**
	Total	120				

Note: *For passing the student has to score a minimum of 45 Marks (CIE+SEE: 20 + 25 or 21 + 24)

Course Prerequisite: C programming concepts.

Course Objectives:

Sl. No.	Course Objectives
1	Understand the object-oriented programming paradigm and Gain knowledge about how information is stored in an object.
2	Understand the capability of a class to rely upon another class and functions.
3	Understand about constructors and their uses.
4	Create and process data in files using file I/O functions.
5	Use the generic programming features of C++ including Exception handling.

Course Outcomes (COs)

CO	Course Outcomes	Highest Level of Cognitive Domain	Working Knowledge
CO1	Understand object-oriented programming concepts and be able to design solutions for a given problem.	L2	WK2, WK4
CO2	Able to reuse code with extensible Class types, user-defined operators and function overloading.	L2	WK3, WK4, WK6
CO3	Achieve code reusability and extensibility by means of Inheritance and Polymorphism	L3	WK3, WK4, WK6, WK8
CO4	Incorporate file-handling functions and exceptions for a given application.	L3	WK3, WK4, WK6, WK7

CO5	Design C++ packages with templates for providing solutions to complex problems.	L3	WK4, WK6, WK8

L1 – Remember, L2 – Understand, L3 – Apply, L4 – Analyze, L5 – Evaluate, L6 – Create

Course Content / Syllabus:

UNIT No.	Content	Hours Lecture
1	**Principles of Object-Oriented Programming:** Software crisis, software evolution, A look at procedure-oriented programming, object-oriented programming paradigm, basic concepts of object-oriented programming, benefits of OOP, object-oriented languages, applications of OOP. **Beginning with C++:** What is C++?; application of C++, a simple C++ program, more C++ statements, an example with Class, structure of C++ program, creating the source file, compiling and linking. **Tokens, Expressions and Control Structures:** Introduction, tokens, keywords, identifiers and constants, basic datatypes, user-defined datatypes, storage classes, derived datatypes, symbolic constants, type compatibility, declaration of variables, dynamic initialization of variables, reference variables, operators in C++, scope resolution operator, member dereferencing operators, memory management operators, manipulators, type cast operators, expressions and their types, special assignment expressions, implicit conversions, operator overloading, operator precedence, control structures. **Functions in C++:** Introduction, the main() function, function prototyping, call by reference, return by reference, inline functions, default arguments, 'const' arguments, recursion, function overloading, friend and virtual functions, Math library functions. **Self-Learning and Term Work*** • Explore the historical evolution from procedural languages (C, Pascal) to OOP and compare with modern paradigms (Python, Java). • Investigate the role of OOP in real-world applications such as GUI frameworks or game development.	09
2	**Classes and Objects:** Introduction, C structure revisited, specify a class, defining member functions, C++ program with Class, making an outside function inline, nesting of member functions, private member functions, arrays within a class, memory allocation for objects, static data members, static member functions, arrays of objects, object as function arguments, friendly functions, returning objects, 'const' members functions, pointers to members. **Constructors and Destructors:** Introduction, constructors, parameterized constructors, multiple constructors in a class, constructor with default arguments, dynamic initialization of objects, copy constructor, dynamic constructors, constructing two-dimensional arrays, destructors. **Operator Overloading and Type Conversions:** Introduction, defining operator overloading, overloading unary and binary operators,	09

	overloading binary operator using friend, manipulation of string using operators, some other operators overloading, rules for overloading operators, type conversion. **Self-Learning and Term Work*** • Explore memory management strategies: stack vs heap allocation in C++ compared to other languages. • Study smart pointers (e.g., unique_ptr, shared_ptr) as an extension to memory management in classes.	
3	**Inheritance: Extending Classes** Introduction, defining derived classes, single inheritance, making a private members inheritable, multilevel inheritance, multiple inheritance, hierarchical inheritance, virtual base classes, abstract classes, constructors in derived classes, member classes: nesting of classes. **Pointers, Virtual Functions and Polymorphism:** Introduction, pointers, pointers to objects, 'this' pointer, polymorphism, pointers to derived classes, virtual functions, pure virtual functions, virtual constructors and destructors. **Self-Learning and Term Work*** • Study the difference between early binding (static polymorphism) and late binding (dynamic polymorphism) and write a simple program using both function overloading and virtual functions to demonstrate each concept. • Explore polymorphism in standard libraries (STL iterators, streams).	09
4	**Managing Console I/O Operations:** introduction, C++ streams, C++ stream classes, unformatted I/O operations, formatted console I/O operations, managing output with manipulators. **Working with Files:** Introduction, classes for file stream operations, opening and closing files, detecting end-of-file. **Exception Handling:** Introduction, basics of exception handling, exception handling mechanism, throwing mechanism, catching mechanism. **Self-Learning and Term Work*** • Develop a console-based C++ application (e.g., a simple text processor), dividing tasks for I/O operations and exception handling. • Custom exception classes in C++	09
5	**Templates:** Introduction, class templates, class templates with multiple parameters, function templates, function templates with multiple parameters, overloading of template functions, member function templates. **Self-Learning and Term Work*** • Explore how templates enable generic programming and influence modern C++ libraries. • Implement a generic swap function using a function template and demonstrate how this single function can be used to swap variables of different data types (e.g., int, double, string) without needing to write multiple functions.	09

*** NOTE: Self-Learning and Term Work are evaluated only through continuous internal evaluation (CIE) and will not be assessed during the Semester End Examination (SEE).**

Text Books:

Sl. No.	Author/s	Title	Publisher Details
1	Herbert Schildt	C++: The Complete Reference	12th Edition, Tata McGraw Hill Education, 5 December 2022, ISBN-13: 978-9355323712
2	E Balagurusamy	Object-Oriented Programming with C++	8th Edition, Tata McGraw Hill Education, 24 September 2020, ISBN-13: 978-9389949186

Reference Books:

Sl. No.	Author/s	Title	Publisher Details
1	Bjarne Stroustrup	C++ PROGRAMMING LANGUAGE	4th Edition, Pearson Education, 31 May 2022, ISBN-13: 978-9356060135
2	Stanley B.Lippmann, Josee Lajore	C++ Primer	4th Edition, Pearson Education, 2005.
3	Yashavant Kanetkar	Let Us C++	BPB Publications Edition, 16 September 2020, ISBN-13: 978-9388176644
4	Behrouz A. Forouzan, Richard F. Gilberg	C++ Programming: An Object-Oriented Approach	First Edition, Tata McGraw Hill Education, 19 May 2022, ISBN-13: 978-9355321305

Web Resources:

Sl. No.	Web Links
1	Programming in C++: https://archive.nptel.ac.in/courses/106/105/106105151/
2	Programming In Modern C++: https://onlinecourses.nptel.ac.in/noc22_cs103/preview

Course Articulation Matrix:

Course Outcomes	Program Outcomes											Program Specific Outcomes		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	3	-	-	-	-	-	-	-	-	-	-	-	-	-
CO2	3	3	3	-	-	-	-	3	3	-	-	3	-	3
CO3	3	3	3	-	3	-	-	3	3	-	-	3	-	3
CO4	3	3	3	1	3	-	-	3	3	-	-	-	-	3
CO5	3	3	3	-	3	-	-	-	-	-	-	3	-	3

High – 3, Medium – 2, Low – 1

LIST OF PROGRAMS FOR PRACTICAL SESSIONS

Weeks	List of Programs	No. of Hours
1	a) Write a C++ program to find and output all the roots of a quadratic equation. b) Write a C++ program to find the sum of all the natural numbers from 1 to n. c) Write a C++ program to sort the elements in ascending and descending order.	02
2	The daily maximum temperatures recorded for 5 cities during the first 6 days of January month have to be tabulated. Write a C++ program to read the data and find the city and day corresponding to highest temperature and lowest temperature.	02
3	a)Write a C++ program to swap 2 values by writing functions for the following: i. Call-by-name/value ii. Call-by-pointers and iii. Call-by-reference b) Define a Cube class and write a C++ program to read the side for three cubes and print their volume and outer area.	02

4	a)An election is contested by 5 candidates. The candidates are numbered 1 to 5 and the voting is done by marking the candidate number on the ballot paper. Develop an application to read the ballots and count the votes cast for each candidate using an array variable count. In case, a number read is outside the range 1 to 5, the ballot should be considered as a 'spoilt ballot' and the program should also count the number of spoilt ballots.
	b) Write a C++ program to demonstrate function overloading for the following prototypes.
	sum_two_nums(int a, int b);
	sum_two_nums(double a, double b);
		02

5	Given that an EMPLOYEE class contains following members: Data members: Employee_Number, Employee_Name, Basic, DA, IT, Net_Salary. Member functions: to read the data, to calculate Net_Salary and to print data members. Write a C++ program to read the data of N employees and compute Net_Salary of each employee. (Dearness Allowance (DA) = 52% of Basic and Income Tax (IT) = 30% of the gross salary. Net_Salary = Basic + DA - IT)	02
6	Define a STUDENT class with USN, Name, and Marks in 3 tests of a subject. Declare an array of 10 STUDENT objects. Using appropriate functions, find the average of two better marks for each student. Print the USN, Name, and the average marks of all the students.	02
7	Write a C++ program to create a class called COMPLEX and implement the following overloading functions ADD that return a	02

	COMPLEX number. i. ADD (a, s2) – where a is an integer (real part) and s2 is a complex number. ii. ADD (s1, s2) – where s1 and s2 are complex numbers.	
8	Create a class named Shape with a function that prints "This is a shape". Create another class named Polygon inheriting the Shape class with the same function that prints "Polygon is a shape". Create two other classes named Rectangle and Triangle having the same function which prints "Rectangle is a polygon" and "Triangle is a polygon" respectively. Again, make another class named Square having the same function which prints "Square is a rectangle". Now, try calling the function by the object of each of these classes. Write a C++ program to demonstrate the same.	02
9	Suppose we have three classes Vehicle, FourWheeler, and Car. The class Vehicle is the base class, the class FourWheeler is derived from it and the class Car is derived from the class FourWheeler. Class Vehicle has a method 'vehicle' that prints 'I am a vehicle', class FourWheeler has a method 'fourWheeler' that prints 'I have four wheels', and class Car has a method 'car' that prints 'I am a car'. So, as this is a multi-level inheritance; we can have access to all the other classes methods from the object of the class Car. We invoke all the methods from a Car object and print the corresponding outputs of the methods. So, if we invoke the methods in this order, car(), fourWheeler(), and vehicle(), then the output will be, • I am a car • I have four wheels • I am a vehicle Write a C++ program to demonstrate multilevel inheritance using the above.	02
10	Write a function which throws a division by zero exception and catch it in catch block. Write a C++ program to demonstrate usage of try, catch and throw to handle exception.	02
11	Write a C++ program function which handles array of bounds exception using C++.	02
12	Write a C++ program that consists of Stack class and include methods to push and pop the stack elements. Create your own defined Exception class and handle exceptions while stacks run with “Stack as full” and “Stack under flow” while calling push() and pop() methods respectively. Implement this using ‘throws’ and ‘throw’ keywords.	02
13	**Revision**	02
14	**Revision**	02
15	**Laboratory Test/Event**	02